

Cross-Candidate Verification for Scientific Software Repair

Nine scientific software tasks · three proposed repairs per task

Repository: [WrettySV/selfverify-swe-science](#) · Reproduction: [README.md](#)

Abstract

A coding agent can make a scientific program pass the visible example while leaving its underlying error unresolved. Extra tests could expose that error, but a model writing tests for its own answer may repeat the assumptions that produced the answer. We study two uses of generated tests: A, revise a repair when its own tests fail; and B, choose among three repairs using the other repairs' tests. On nine SWE-bench Science tasks, the baseline solves 10/27 attempts (37.04%). A returns 12/27 correct answers (44.44%). B improves single-answer selection success from 44.44% for random choice of those same starting answers to 50.00%. Both baseline and the A/B starting sets contain a correct answer on five of nine tasks. The comparison shows that cross-testing can improve the choice among available repairs, while the five-task coverage limits the best possible selection result. A complete-solver comparison at equal inference cost remains open.

1. Why test a repair with another agent's tests?

Scientific software must preserve relationships that a single example may not exercise: invariance to coordinate changes, consistent units, conservation laws, or sensible behavior at a boundary. A patch can fix the displayed output while violating another such relationship. Additional executable tests can turn these requirements into concrete feedback. Their value depends on whether they distinguish correct and incorrect behavior, rather than merely agree with a plausible patch.

SWE-bench Science contains 119 tasks from 98 repositories across 20 scientific domains. Agents can use public checks during repair; hidden scientific checks assess the resulting solution. The benchmark documents superficial repairs and failures to generalize beyond observed cases. [Xu et al., 2026](#) Our question is: **do generated tests expose errors well enough to improve the same repair, or to choose a better repair from several attempts?**

Self-Refine studies feedback followed by revision of the same output. [Madaan et al., 2023](#) Findings that intrinsic self-correction can fail motivate examining the source and reliability of feedback. [Huang et al., 2024](#) CodeT uses generated tests to select code solutions. [Chen et al., 2022](#) CodeMonkeys combines multiple repository-repair trajectories, generated tests and a selection trajectory; Kozuchi directly tests repairs using other runs' checks. [Ehrlich et al., 2025](#), [Bahrami et al., 2026](#)

Building on these approaches, we implement a small cross-testing system for scientific repositories and compare its selection evidence with the feedback available for own-test refinement. We request tests of scientific behavior, exclude a repair's own tests from its selection score, and separate failed assertions from execution errors. Our contribution is this implementation and empirical analysis of when its generated feedback helps or fails. The cross-testing principle itself is established by prior work.

2. How the method handles one task

Three proposed repairs. For each task, verification starts with three independently produced repairs. Each test-writing session receives one repair in a separate repository copy and creates Python checks of the intended scientific behavior. Test writers do not see the other repairs. The controller keeps the generated test code unchanged while running it on all three repairs in clean repository copies. This creates a three-by-three comparison of repairs and test sets.

A: use own tests to improve an answer. If a repair fails its own executable checks, the agent receives their failures and may revise the repair once. It then reruns the same checks. A revision is retained only if results improve without breaking previously passing checks. Passing own tests leave the answer unchanged; missing or invalid tests also leave it unchanged.

B: use other repairs' tests to choose an answer. Each test set gives a positive vote when all its checks pass, or a negative vote when an assertion fails. A repair's own test set is excluded. A set can vote only if it exposes an assertion failure in the original unfixed program without technical errors there. A timeout or import error on a proposed repair gives no vote for that comparison. B prefers public-passing repairs and selects the highest average foreign-test vote. Equally scored winners are chosen uniformly at random; if no usable evidence exists, B chooses randomly among public-passing repairs.

A and B share the same three starting repairs and generated tests. B selects among those starting repairs; it does not consume A's revised outputs. Hidden benchmark scores are used to assess correctness after selection, never to compute votes or choose a repair.

3. Tasks, implementation and metrics

We evaluate tasks **001, 007, 008, 017, 024, 039, 053, 094 and 107**, spanning reaction chemistry, MR spectroscopy, plasma physics, gamma-ray astronomy, probabilistic computing, Earth-surface dynamics, medical imaging, magnetic symmetry and computer algebra. The subset uses a common Python execution path and one task per repository. From the initial panel, 033 is excluded for high repeated-execution cost, 004 requires a native rebuild path, and 014 duplicates the repository and base commit of 008. All other eligible tasks remain, including tasks with no correct repair. The tasks had been inspected during development, so this is an exploratory subset.

The baseline has three attempts per task. A/B use a separate set of three existing repairs per task, with newly generated tests. We report A's final outcomes and compare B with random selection from the same repairs. Baseline versus A/B is an additional descriptive comparison; their starting repairs and total generation costs are not matched. Detailed input identities are provided with the artifacts.

Inference uses **Qwen3.8-27B served locally with vLLM 0.28.0**, BF16 weights, FP8 KV cache and a 262,144-token context window. The agent runtime is Codex 0.154.0 through Pier 0.3.0, with xhigh reasoning. Task execution uses Docker environments. The model revision, generation limits and recorded token usage are specified in the reproduction guide and configuration artifacts.

An answer is correct if the benchmark's hidden verifier returns score 1. **Mean pass@1** for baseline and A is the correct fraction within each task, averaged over tasks. **Coverage within three answers** records whether at least one of the three is correct: the observed pass@3 event, which need not imply successful selection. [Chen et al., 2021](#) For B we calculate the fraction of correct repairs among equally scored winners and average across tasks. This measures one-answer selection success on the available repairs. Repeated complete B runs, and hence B pass@3, are not measured.

4. What was solved?

Table 1. Final answers from baseline and A. S = correct; F = incorrect. The columns contain different repair attempts; positions are not paired across methods.

Task	Baseline attempts	A: own-test verification
001	F, F, F; 0/3	F, F, F; 0/3
007	F, F, S; 1/3	F, S, F; 1/3
008	F, F, F; 0/3	F, F, F; 0/3
017	S, F, S; 2/3	S, S, S; 3/3
024	S, S, F; 2/3	S, S, S; 3/3
039	F, F, F; 0/3	F, F, F; 0/3
053	S, F, S; 2/3	S, S, F; 2/3
094	F, F, F; 0/3	F, F, F; 0/3
107	S, S, S; 3/3	S, S, S; 3/3
Correct answers	10/27	12/27
Mean pass@1	37.04%	44.44%
Tasks with at least one correct answer	5/9 (55.56%)	5/9 (55.56%)

Table 2. Success of one answer, by task. Baseline and A are empirical pass@1. Random choice selects from B's three starting repairs; all pass the public test. B averages over uniformly chosen best-scoring repairs. The columns describe the same outcome, correctness of one answer, but only the random-choice/B comparison uses identical inputs.

Task	Baseline	Random choice	A	B
001	0.00%	0.00%	0.00%	0.00%
007	33.33%	33.33%	33.33%	50.00%
008	0.00%	0.00%	0.00%	0.00%
017	66.67%	100.00%	100.00%	100.00%
024	66.67%	100.00%	100.00%	100.00%
039	0.00%	0.00%	0.00%	0.00%
053	66.67%	66.67%	66.67%	100.00%
094	0.00%	0.00%	0.00%	0.00%
107	100.00%	100.00%	100.00%	100.00%
Mean	37.04%	44.44%	44.44%	50.00%

B's mean is $(0 + 50 + 0 + 100 + 100 + 0 + 100 + 0 + 100) / 9 = 50\%$. It leaves only correct answers on four tasks, a correct/incorrect tie on 007, and only incorrect answers on four tasks. A single final draw would solve four or five tasks; 50% averages over the unresolved tie. Both baseline and A have at least one correct answer on **007, 017, 024, 053 and 107**, giving observed three-answer coverage of 5/9.

5. What do these results tell us?

Cross-testing helps choose between some answers. On 007, the starting repairs are F, S, F. Their foreign-test scores are 1/2, 1/2 and 0/2. B excludes the third repair but cannot distinguish the first two: one is correct, so selection success rises from 1/3 to 1/2. On 053, B selects a correct repair and raises success from 2/3 to 1. Across tasks this is a **5.56 percentage-point gain over random choice of the same repairs**.

B exceeds the baseline's 37.04% by 12.96 points, but those results use different starting repairs and unmatched total costs. The 5.56-point comparison with random choice uses identical repairs and isolates the effect of the selection rule.

Availability of generated tests. Nineteen of 27 test-generation attempts produced scripts in the required format. Each accepted set was run on all three repairs for its task. The remaining eight outputs lacked explicit assertions or a valid list of test files. These requirements check the format of the tests, not whether their scientific expectations are correct; they can also reject useful checks written in another form. On 024, no test set was accepted. B still succeeds there because all three repairs are correct, so any fallback choice succeeds.

Limits of the evidence. Only 007 and 053 have both correct and incorrect A/B inputs. Four tasks have no correct input and three have only correct inputs. Thus most tasks cannot demonstrate a selection gain. Nine previously inspected tasks and one three-repair comparison per task provide limited evidence about generalization. Excluding development task 017 gives 33.33% for baseline, 37.50% for random choice and A, and 43.75% for B.

Generated tests may express incorrect scientific expectations. An assertion failure is executable evidence, but does not prove the assertion is valid; the 007 example includes a correct patch rejected by a generated test set. Excluding own votes also cannot remove assumptions shared by the model across branches. In the implementation, foreign patch files are accessible before a possible A revision; a stricter A evaluation should isolate that workspace.

A stronger follow-up would generate all starting repairs under one protocol, repeat complete runs, match the total inference allowance across methods, and grade one returned patch per run.

6. Reproduction and conclusion

The README provides one entry point, `python3 scripts/run_study.py`, that reconstructs the tables from supplied results. The same controller can generate new tests, cross-test the supplied repairs, attempt own-test revision and report the resulting metrics. Patch identities, test files, outcomes, selection code and revision records are included. Model weights are referenced by their published repository and revision.

Cross-testing raises mean selection success from 44.44% to 50.00% on the available repairs, with the gain concentrated on the two tasks containing both correct and incorrect answers. This supports further study of generated tests as a selection signal. The next step is a broader comparison of complete methods at equal inference cost.

AI disclosure. Qwen3.8-27B is the evaluated model. AI assistance included Cursor (Auto mode) for code development and DeepSeek for analysis, bug investigation and report writing.

References

1. Zhipeng Xu, Jiahao Lu, Yining Zheng, Yuxin Wang, and Xipeng Qiu. **SWE-bench Science: Can Coding Agents Resolve Engineering Tasks in Science?** 2026. [arXiv:2608.19799](#). Dataset: [OpenMOSS-Team/SWE-bench-Science](#).
2. Aman Madaan et al. **Self-Refine: Iterative Refinement with Self-Feedback**. 2023. [arXiv:2303.17651](#).
3. Jie Huang et al. **Large Language Models Cannot Self-Correct Reasoning Yet**. ICLR 2024. [arXiv:2310.01798](#).
4. Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. **CodeT: Code Generation with Generated Tests**. 2022. [arXiv:2207.10397](#).
5. Ryan Ehrlich, Bradley Brown, Jordan Juravsky, Ronald Clark, Christopher Ré, and Azalia Mirhoseini. **CodeMonkeys: Scaling Test-Time Compute for Software Engineering**. 2025. [arXiv:2501.14723](#).
6. Mehdi Bahrani et al. **Kozuchi Agent: A Language-Agnostic Open-Weight Agent for Software Repair**. 2026. [arXiv:2608.15579](#).
7. Mark Chen et al. **Evaluating Large Language Models Trained on Code**. 2021. [arXiv:2107.03374](#).
8. Qwen Team. **Qwen3.8-27B model weights and configuration**. [Model repository](#), pinned revision.